Text Compression Using Character Runs

1. Problem Statement

Write a Python program to compress a given string by grouping consecutive repeated characters into a character-and-count format. The program should count the number of consecutive occurrences of each character and represent them as a character followed by its count. Finally, compare the length of the original string with the compressed string and display both lengths.

For example, the string aaabbccccd is compressed into a3b2c4d1.

2. Objectives

1. To understand string manipulation in Python.
2. To identify consecutive repeated characters in a string.
3. To count repeated characters using loops and counters.
4. To generate a compressed string using character-and-count groups.
5. To compare the lengths of the original and compressed strings.

3. Assumptions

1. The input contains only English letters and does not contain spaces.
2. The program is case-sensitive; uppercase and lowercase letters are treated as different characters.
3. Only consecutive repeated characters are grouped together.
4. Every character is followed by its count, even when the count is 1.
5. The original string is not empty.
6. The compressed length is calculated by counting the characters in the compressed representation.

4. Algorithm

1. Start the program.
2. Read a string from the user.

3. Initialize an empty string named compressed.
4. Initialize a counter count to 1.
5. Traverse the input string from the second character.
6. If the current character is equal to the previous character, increase the counter by 1.
7. Otherwise, append the previous character and its count to the compressed string, then reset the counter to 1.
8. After the loop, append the last character and its count.
9. Calculate the length of the original string and the compressed string.
10. Display the original string, compressed string, and their lengths.
11. Stop the program.

5. Flowchart

6. Pseudocode

```
BEGIN

    INPUT original


    compressed = ""
    count = 1
```

```
    FOR i = 1 TO length(original) - 1

        IF original[i] == original[i - 1] THEN
            count = count + 1

        ELSE
            compressed = compressed + original[i - 1] + count
            count = 1
        END IF

    END FOR

    compressed = compressed + last character + count

    original_length = length(original)
    compressed_length = length(compressed)

    PRINT original
    PRINT compressed
    PRINT original_length
    PRINT compressed_length

END
```

7. Python Actual Code

```
# Text Compression Using Character Runs

original = input("Enter a string: ")

compressed = ""
count = 1

for i in range(1, len(original)):
    if original[i] == original[i - 1]:
        count += 1
    else:
        compressed += original[i - 1] + str(count)
        count = 1

# Add the final character and its count
if original:
    compressed += original[-1] + str(count)

# Calculate lengths
original_length = len(original)
compressed_length = len(compressed)

# Display results
print("Original String:", original)
print("Compressed String:", compressed)
print("Original Length:", original_length)
print("Compressed Length:", compressed_length)

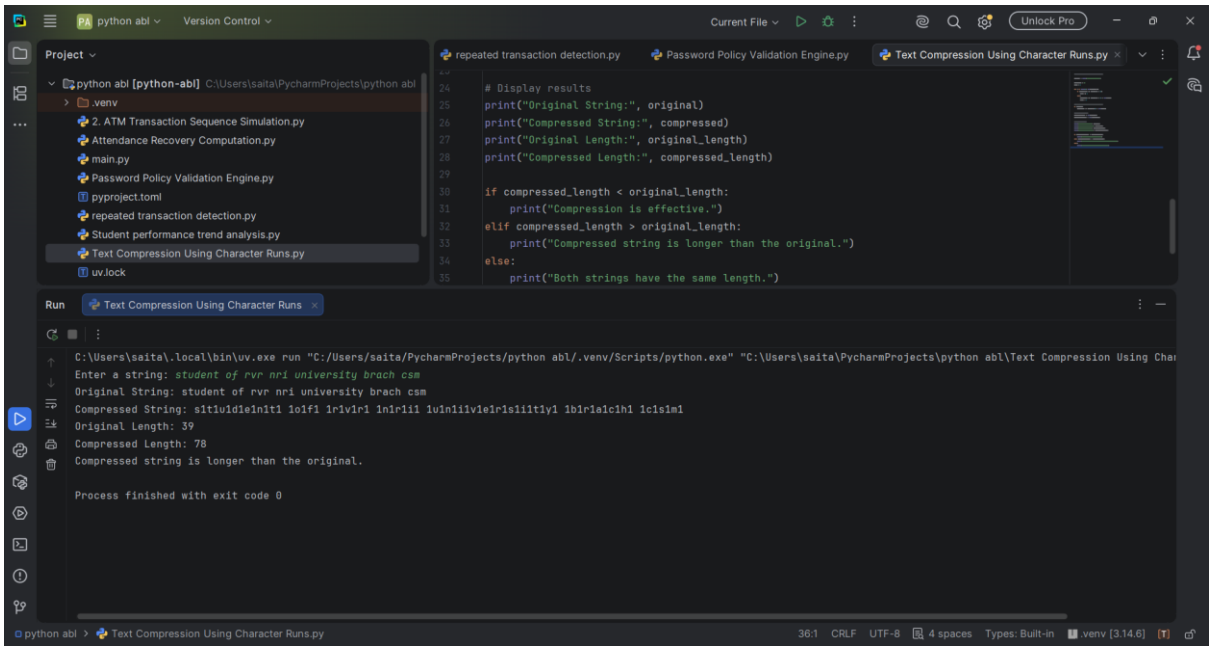
if compressed_length < original_length:
    print("Compression is effective.")
elif compressed_length > original_length:
    print("Compressed string is longer than the original.")
```

```
else:
    print("Both strings have the same length.")
```

8. Explanation of the Code

- original = input(...): Reads the string entered by the user.
- compressed = "": Creates an empty string to store the compressed result.
- count = 1: Initializes the count for the first character.
- for i in range(1, len(original)): Visits each character starting from the second character.
- if original[i] == original[i - 1]: Checks whether the current character matches the previous character.
- count += 1: Increases the count when consecutive characters are repeated.
- compressed += original[i - 1] + str(count): Adds the previous character and its count to the compressed string when a different character is found.
- count = 1: Resets the counter for the new character.
- original[-1]: Accesses the last character of the input string.
- len(original) and len(compressed): Calculate the lengths of the two strings.
- The final if-elif-else statements compare the lengths and display whether compression is effective.

9. Output



10. Conclusion

The Text Compression Using Character Runs program demonstrates how Python strings, loops, counters, and conditional statements can be used to compress consecutive repeated characters. It generates a character-and-count representation and compares the lengths of the original and compressed strings. This program helps beginners understand basic data compression techniques. However, compression is not always effective because strings containing many non-repeated characters may become longer after compression.